Методичка:
https://docs.google.com/document/d/1kCMYYeqlH2Pf8-wOVhH3Y1CW-TW0wh_kCZQjF8XSt5U/edit?usp=sharing

https://github.com/users/Elizavetaist12/projects/1/views/1

MealPlan — карточка проекта

Название: MealPlan — система планирования недельного рациона и формирования списка продуктов

Проблема: людям сложно планировать домашнее питание на неделю: меню придумывается хаотично, список продуктов составляется вручную, часть продуктов покупается лишней или портится.

Целевая аудитория: Занятые люди 20–45 лет, пары и семьи, которые готовят дома и хотят экономить время и деньги; gym girl/boy, следящие за питанием.

Заказчик: Условный заказчик учебного проекта — пользователь, которому необходимо упростить планирование питания и покупок.

Цель проекта: Разработать MVP информационной системы, позволяющей создавать и просматривать рецепты, составлять недельное меню и автоматически формировать список покупок.

Ожидаемый результат: Рабочая демонстрационная версия системы с основными пользовательскими сценариями.

Ограничения: 4 человека в команде; ограниченное время (семестр); только MVP; веб-версия; без реальных платежей и доставки; стартовый каталог 10-20 рецептов;

Границы проекта: В проект входят рецепты, недельное меню и список покупок. Не входят заказ и доставка продуктов, платежи, интеграция с магазинами, медицинские рекомендации и сложная персонализация питания.

Условия приёмки:

- Пользователь может создать рецепт с названием и ингредиентами.
- Созданный рецепт отображается в списке рецептов.
- Пользователь может просматривать список сохраненных рецептов.
- Пользователь может добавить блюдо в меню на выбранный день.
- Система формирует список покупок на основе выбранных блюд.
- Список можно редактировать, отмечать купленное
- Основные сценарии проверены, критические ошибки отсутствуют.

- Есть демо-сценарий и README.

Условие завершения проекта: MVP демонстрирует все обязательные сценарии, проходит проверку и готов к презентации преподавателю.

Роли на проекте
Елизавета - PM, QA - специалист
Анна - аналитик, UI/UX дизайнер
Ангелина - Backend - Разработчик
Елена - Frontend - Разработчик

1. PM + QA — руководитель проекта и тестировщик

- Планирует сроки и контролирует выполнение задач.
- Определяет приоритеты совместно с заказчиком.
- Ведёт backlog и контролирует milestones.
- Проверяет выполнение критериев приёмки.
- Тестирует основные пользовательские сценарии и регистрирует баги.

2. Backend-разработчик

- Проектирует структуру данных.
- Реализует сохранение и получение рецептов.
- Реализует логику меню и формирования списка покупок.

3. Frontend-разработчик

- Разрабатывает формы рецептов.
- Создаёт каталог и интерфейс недельного меню.
- Реализует отображение списка покупок.

4. Бизнес-аналитик + UI/UX-дизайнер

- Описывает требования и пользовательские сценарии.
- Формирует критерии приёмки совместно с PM.
- Создаёт простые макеты экранов и продумывает удобство интерфейса.
- Уточняет ожидаемое поведение системы.

Кто какие решения принимает

Решение	Ответственный
Определение приоритетов задач	PM с учётом требований заказчика

Подтверждение завершения проекта	Заказчик - преподаватель
Подтверждение завершения задачи	PM после проверки результата ответственным исполнителем
Изменение состава релиза	PM согласует с заказчиком
Изменение приоритета задач	PM
Приоритизация найденных ошибок	PM совместно с разработчиком и тестировщиком
Технический способ реализации	Ответственный разработчик с учётом согласованных требований

6. Статусы задача на GIt

Backlog - Запланировано, Ready - Готов к выполнению, In progress - Выполняется, Done - Завершено

7.Backlog — это упорядоченный список работ, необходимых для создания продукта. Каждая задача должна описывать конкретный результат, а не просто идею.

Эпики — крупные блоки работ, к которым относятся задачи.

Backlog MealPlan

20 задач

Эпик 1. Аналитика и подготовка

ID	Задача	Основной исполнитель
MP-01	Определить требования к системе	Аналитик
MP-02	Описать пользовательские сценарии	Аналитик
MP-03	Определить критерии приёмки	PM + Аналитик
MP-04	Спроектировать и развернуть базу данных для рецептов	Backend
MP-05	Подготовить README.md и описание проекта	PM
MP-20	Настроить доступы участников команды в GitHub	PM

MP-22	Создать карту пользовательского пути	UI

Эпик 2. Управление рецептами

ID	Задача	Основной исполнитель
MP-06	Создать базовую структуру приложения	Frontend
MP-07	Создать форму добавления рецепта	Frontend + UI
MP-08	Реализовать функцию сохранение рецепта	Backend
MP-10	Разработать API для получения списка рецептов	Backend
MP-09	Реализовать просмотр списка рецептов	Frontend
MP-26	Реализовать просмотр отдельного рецепта	Frontend
MP-11	Добавить блюдо в меню на выбранный день	Frontend
MP-23	Разработать API для получения списка рецептов	Backend

Для MP-10 разработчику интерфейса тоже понадобится поучаствовать, но за основную реализацию и сохранение изменений отвечает Backend.

Эпик 3. Меню и список покупок

ID	Задача	Основной исполнитель

MP-13	Реализовать просмотр меню на неделю	Frontend + UI
MP-14	Реализовать сбор ингредиентов из меню	Backend
MP-15	Реализовать API для формирования списка покупок	Backend
MP-16	Добавить отметку купленных продуктов	Frontend
MP-24	Реализовать API и хранение меню на неделю	Backend

Эпик 4. Тестирование и выпуск

ID	Задача	Основной исполнитель
MP-17	Определить подход к тестированию и ведению документации MVP	QA
MP-18	Протестировать каталог и просмотр рецепта	QA
MP-21	Протестировать создание и сохранение рецепта	QA
MP-24	Проверить добавление блюд в недельное меню	QA
MP-25	Протестировать формирование списка покупок	QA

8. Эпики

Эпик	Задачи	Результат
Аналитика и подготовка	MP-01–MP-05	Требования, критерии приёмки, структура проекта
Управление рецептами	MP-06–MP-11	Рабочий каталог рецептов
Меню и список покупок	MP-12–MP-16	Недельное меню и список покупок
Тестирование и выпуск	MP-17–MP-20	Проверенный MVP и демонстрация

В GitHub Projects эпики можно представить отдельными Issues с меткой `Epic`, а задачи связывать с ними через поле, метки или описание. Для простого учебного проекта достаточно меток `Epic: Аналитика`, `Epic: Рецепты`, `Epic: Меню`, `Epic: Выпуск`, если вы явно показываете группировку.

Шаг 9. Создаём 4 milestones

Milestone — это контрольная точка, то есть значимый промежуточный результат, а не отдельная задача и не колонка доски.

Создавай их в репозитории: Issues → Milestones → New milestone. Названия и описания:

M1 — Требования согласованы

Описание: определены требования, пользовательские сценарии, критерии приёмки и структура данных.

Связанные задачи: MP-01–MP-04.

M2 — Каталог рецептов готов

Описание: пользователь может создать, сохранить и просмотреть рецепт.

Связанные задачи: MP-06–MP-09.

M3 — Меню и список покупок готовы

Описание: блюда добавляются в недельное меню, ингредиенты формируют список покупок.

Связанные задачи: MP-12–MP-15.

M4 — MVP проверен и готов к демонстрации

Описание: основные сценарии проверены, критические ошибки исправлены, подготовлена демонстрация.

Связанные задачи: MP-17–MP-20.

Чек-лист сдачи ЛР1

Тема проекта и сформулированная проблема
Карточка проекта с целью, аудиторией, ограничениями и критериями приёмки
Описаны 4 пользовательских сценария
Зафиксированы границы проекта и исключённые функции
Распределены роли и полномочия участников
Создана доска GitHub Projects с обязательными статусами
Создано 15–25 конкретных задач с приоритетами и исполнителями
Задачи сгруппированы в 3–6 эпиков
Созданы 2–4 milestones
Подготовлено объяснение управленческих решений и ответы на вопросы

Ответы на контрольные вопросы

1. Чем задача проекта отличается от идеи или пожелания?

Идея описывает общее намерение, например «хочу упростить покупки продуктов». Задача определяет конкретную работу и проверяемый результат, например «реализовать формирование списка покупок на основе ингредиентов блюд недельного меню». Задачу можно оценить, назначить исполнителю и проверить.

2. Почему слишком крупная задача плохо управляется?

У неё трудно оценить трудоёмкость и срок, непонятно, кто отвечает за отдельные части, сложно отслеживать прогресс и обнаруживать проблемы. Поэтому крупные работы разделяют на небольшие задачи с конкретными результатами.

3. Какие последствия имеет отсутствие явных границ проекта?

Появляется неконтролируемое расширение объёма работ: новые функции добавляются без пересмотра сроков и ресурсов. В результате команда может не

успеть завершить обязательную функциональность и подготовить MVP к демонстрации.

4. Что должен видеть руководитель проекта на доске за 30 секунд?

- Какие задачи запланированы, выполняются и завершены.
- Кто отвечает за каждую задачу.
- Какие задачи имеют высокий приоритет.
- Есть ли задачи, застрявшие в работе или проверке.
- Насколько проект близок к очередной контрольной точке.

Как объяснить преподавателю два исключённых решения

Можно сказать так:

1. Интеграция с магазинами и доставка продуктов. Не включена, потому что требует внешних API, согласования с поставщиками и дополнительного тестирования. Для проверки основного сценария MealPlan она не нужна.
2. Персональные медицинские рекомендации и расчёт сложных показателей питания. Не включены, потому что значительно расширяют требования и сложность проверки. Основная цель проекта — планирование меню и автоматическое составление списка покупок.